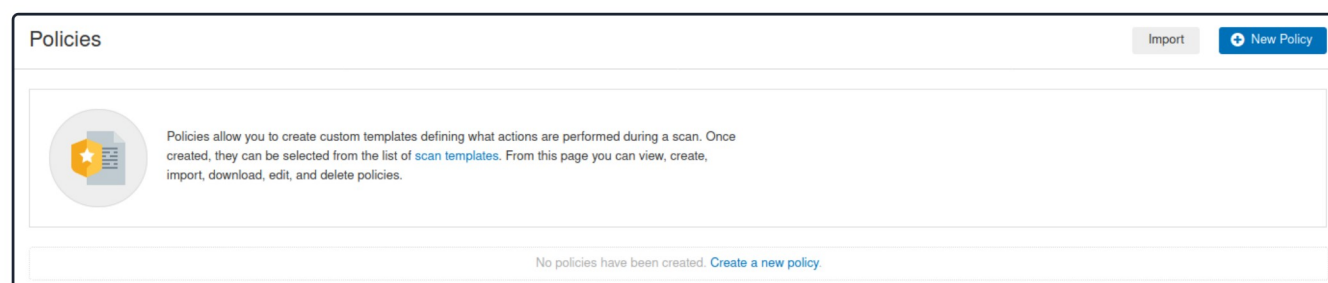


Advanced Settings

We can configure a number of advanced settings for Nessus and its scans, like scan policies, plugins, and credentials, all of which we will cover in this section.

Scan Policies

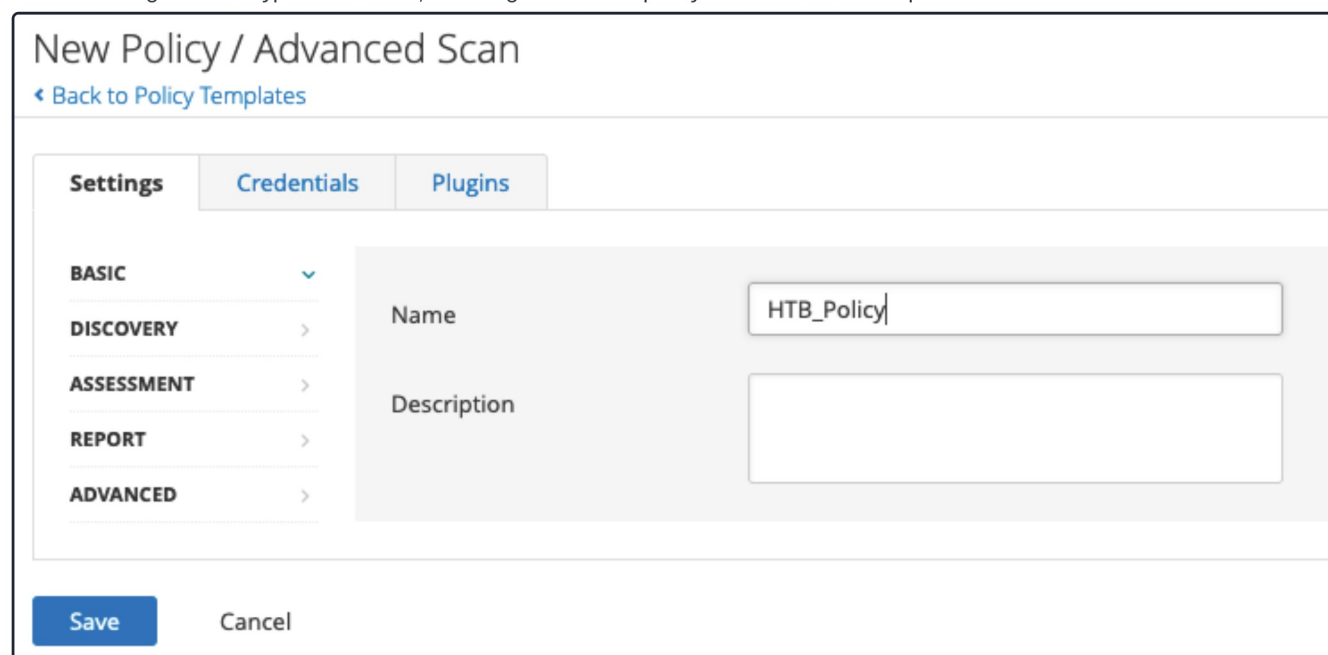
Nessus gives us the option to create scan policies. Essentially these are customized scans that allow us to define specific scan options, save the policy configuration, and have them available to us under **Scan Templates** when creating a new scan. This gives us the ability to create targeted scans for any number of scenarios, such as a slower, more evasive scan, a web-focused scan, or a scan for a particular client using one or several sets of credentials. Scan policies can be imported from other Nessus scanners or exported to be later imported into another Nessus scanner.



Creating a Scan Policy

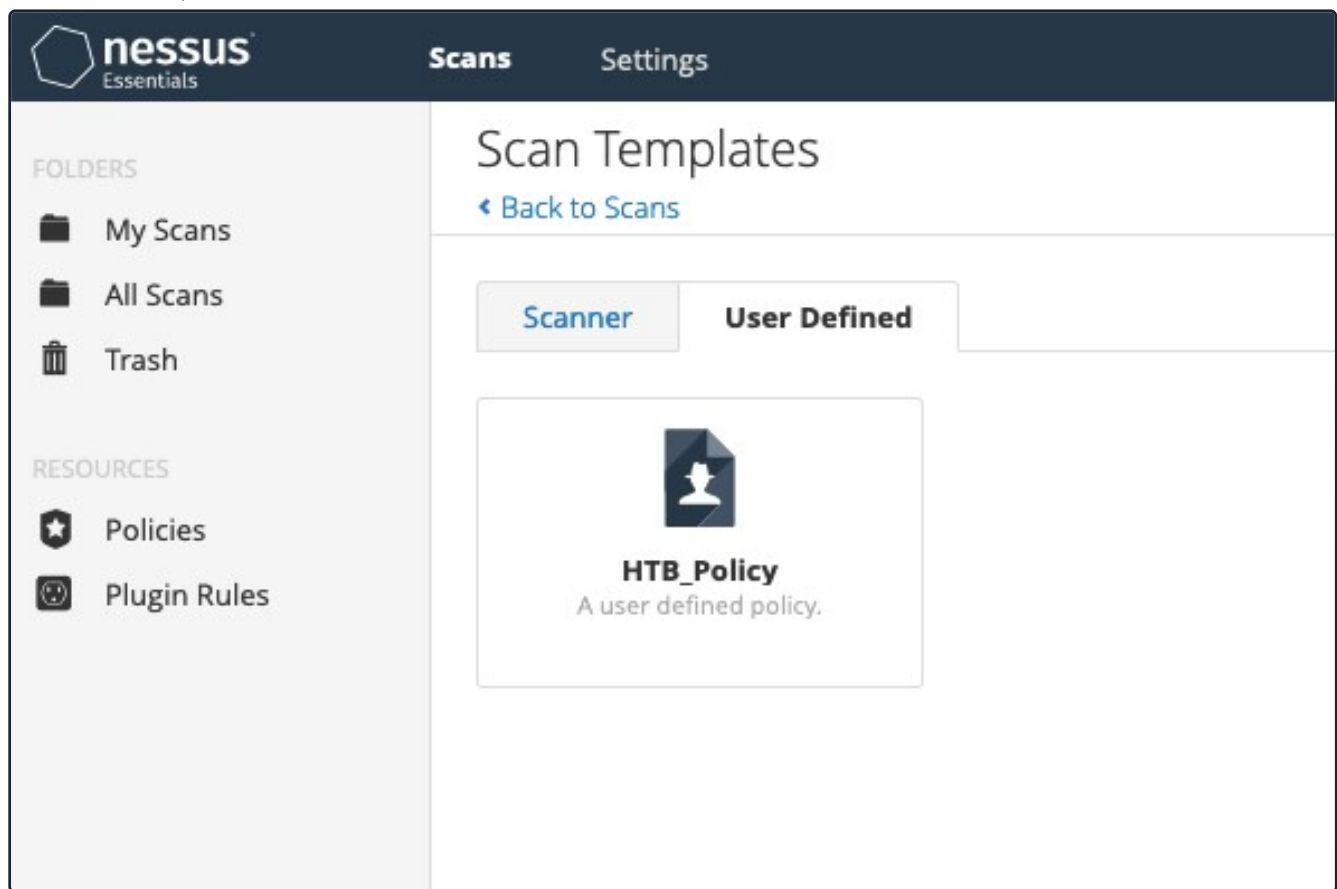
To create a scan policy, we can click on the **New Policy** button in the top right, and we will be presented with the list of pre-configured scans. We can choose a scan, such as the **Basic Network Scan**, then customize it, or we can create our own. We will choose **Advanced Scan** to create a fully customized scan with no pre-configured recommendations built-in.

After choosing the scan type as our base, we can give the scan policy a name and a description if needed:



From here, we can configure settings, add in any necessary credentials, and specify any compliance standards to run the scan against. We can also choose to enable or disable entire **plugin** families or individual plugins.

Once we have finished customizing the scan, we can click on [Save](#), and the newly created policy will appear in the policies list. From here on, when we go to create a new scan, there will be a new tab named [User Defined](#) under [Scan Templates](#) that will show all of our custom scan policies:



Nessus Plugins

Nessus works with plugins written in the [Nessus Attack Scripting Language \(NASL\)](#) and can target new vulnerabilities and CVEs. These plugins contain information such as the vulnerability name, impact, remediation, and a way to test for the presence of a particular issue.

Plugins are rated by severity level: [Critical](#), [High](#), [Medium](#), [Low](#), [Info](#). At the time of this writing Tenable has published [145,973](#) plugins that cover [58,391](#) CVE IDs and [30,696](#) [Bugtraq](#) IDs. A searchable database of all published plugins is on the [Tenable website](#).

The [Plugins](#) tab provides more information on a particular detection, including mitigation. When conducting recurring scans, there may be a vulnerability/detection that, upon further examination, is not considered to be an issue. For example, Microsoft DirectAccess (a technology that provides internal network connectivity to clients over the Internet) allows insecure and null cipher suites. The below scan performed with [ssllscan](#) shows an example of insecure and null cipher suites:

```
dbcypth0n@htb[/htb]$ ssllscan example.com
```

```
<SNIP>
```

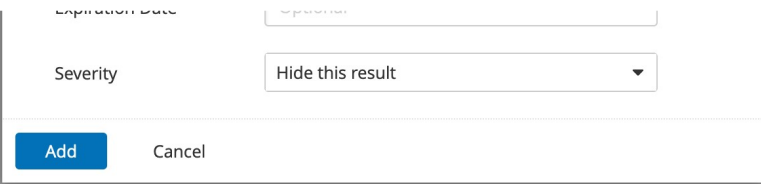
Preferred	TLSv1.0	128 bits	ECDHE-RSA-AES128-SHA	Curve 25519 DHE 253
Accepted	TLSv1.0	256 bits	ECDHE-RSA-AES256-SHA	Curve 25519 DHE 253
Accepted	TLSv1.0	128 bits	DHE-RSA-AES128-SHA	DHE 2048 bits
Accepted	TLSv1.0	256 bits	DHE-RSA-AES256-SHA	DHE 2048 bits
Accepted	TLSv1.0	128 bits	AES128-SHA	
Accepted	TLSv1.0	256 bits	AES256-SHA	

```
<SNIP>
```

However, this is by design. SSL/TLS is not **required** in this case, and implementing it would result in a negative performance impact. To exclude this false positive from the scan results while keeping the detection active for other hosts, we can create a plugin rule:

Under the **Resources** section, we can select **Plugin Rules**. In the new plugin rule, we input the host to be excluded, along with the Plugin ID for Microsoft DirectAccess, and specify the action to be performed as **Hide this result**:

We may also want to exclude certain issues from our scan results, such as plugins for issues that are not directly exploitable (e.g., [SSL Self-Signed Certificate](#)). We can do this by specifying the plugin ID and host(s) to be excluded:



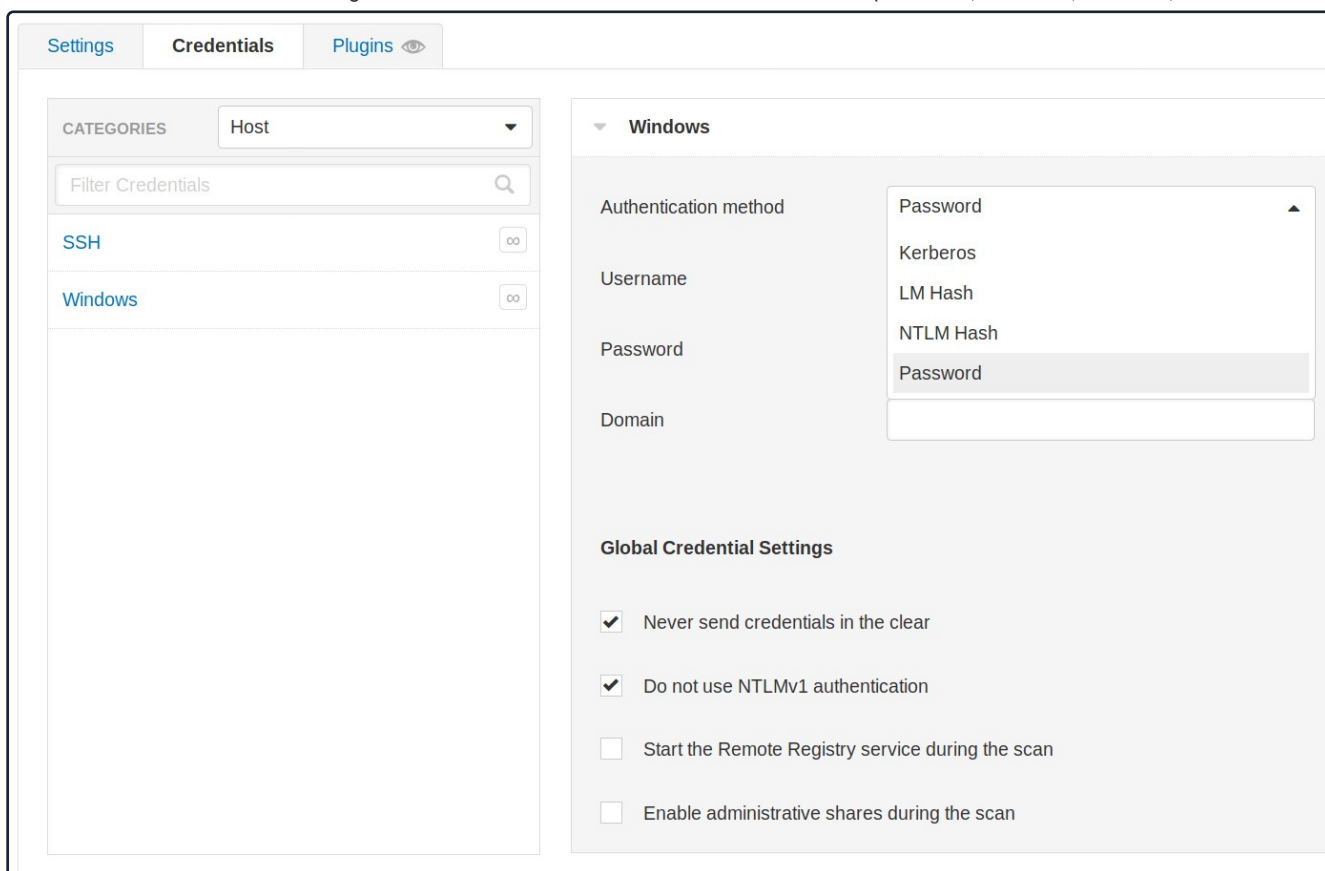
Expiration Date

Severity

Scanning with Credentials

Nessus also supports credentialed scanning and provides a lot of flexibility by supporting LM/NTLM hashes, Kerberos authentication, and password authentication.

Credentials can be configured for host-based authentication via SSH with a password, public key, certificate, or Kerberos-based authentication. It can also be configured for Windows host-based authentication with a password, Kerberos, LM hash, or NTLM hash:



Settings **Credentials** **Plugins**

CATEGORIES Host

Filter Credentials

SSH

Windows

Windows

Authentication method Password

Username

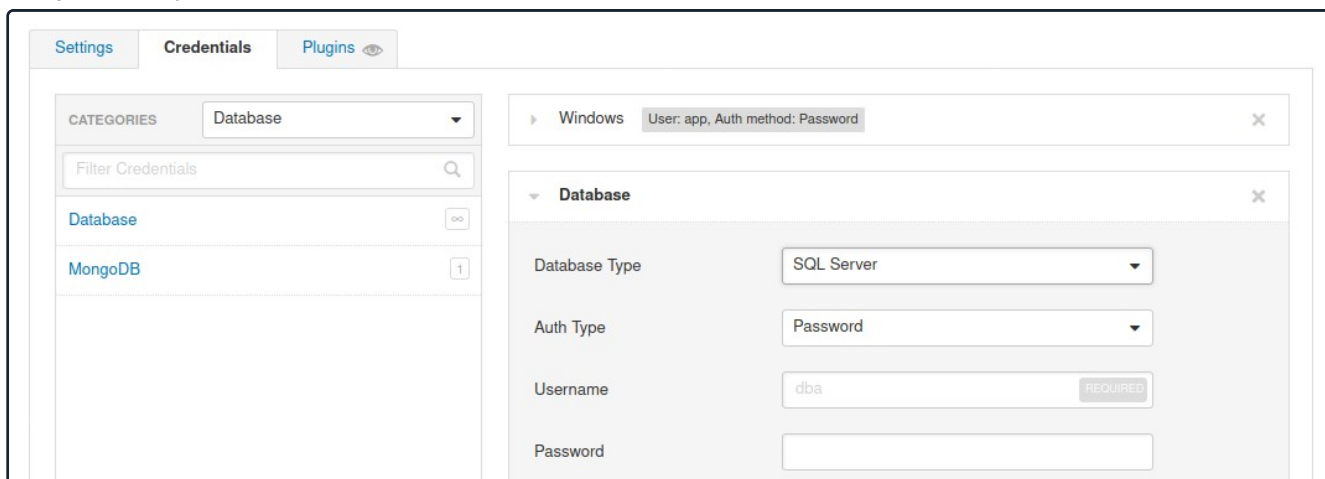
Password

Domain

Global Credential Settings

- ☒ Never send credentials in the clear
- ☒ Do not use NTLMv1 authentication
- ☐ Start the Remote Registry service during the scan
- ☐ Enable administrative shares during the scan

Nessus also supports authentication for a variety of databases types including Oracle, PostgreSQL, DB2, MySQL, SQL Server, MongoDB, and Sybase:



Settings **Credentials** **Plugins**

CATEGORIES Database

Filter Credentials

Database

MongoDB

Windows User: app, Auth method: Password

Database

Database Type SQL Server

Auth Type Password

Username dba REQUIRED

Password

Database Port	1433
Auth type	Windows
Instance name	
If no instance name is provided, the default instance is used.	

Note: To run a credentialed scan on the target, use the following credentials: `htb-student_adm:HTB_@cademy_student!` for Linux, and `administrator:Academy_VA_adm1!` for Windows. These scans have already been set up in the Nessus target to save you time.

In addition to that, Nessus can perform plaintext authentication to services such as FTP, HTTP, IMAP, IPMI, Telnet, and more:

The screenshot shows the Nessus 'Credentials' configuration page. On the left, a list of categories includes FTP, IMAP, IPMI, NNTP, POP2, POP3, SNMPv1/v2c, and telnet/rsh/rexec. The 'HTTP' category is selected. The main configuration area for HTTP shows the following settings:

- Authentication method: HTTP login form
- Username: admin (REQUIRED)
- Password: (REQUIRED)
- Login page: /login.php (REQUIRED)
- Login submission page: /process_login.php (REQUIRED)
- Login parameters: user=%USER%&pass=%PASS% (REQUIRED)
- Check authentication on page: /user/profile.php (REQUIRED)
- Regex to verify successful authentication: Logged in as user "[*"]+" (REQUIRED)

Finally, we can check the Nessus output to confirm whether the authentication to the target application or service with the supplied credentials was successful:

INFO Microsoft SQL Server Login Possible

Description
Nessus was able to log into the remote MS SQL server using the supplied credentials.

See Also
https://azure.microsoft.com/en-us/?ocid=cloudplat_hp

Output

```
Credentialed checks have been enabled for MSSQL server on port 1433.
SQL Server Version    : 14.0.1000.0
SQL Server Instance   :
```

[< Previous](#)[Next >](#)[✔ Mark Complete & Next](#)

Table of Contents

Security Assessments

Security Assessments	✔
Vulnerability Assessment	✔
Assessment Standards	✔

Vulnerability Scoring and Reporting

Common Vulnerability Scoring System (CVSS)	✔
Common Vulnerabilities and Exposures (CVE)	✔

Nessus

Vulnerability Scanning Overview	✔
Getting Started with Nessus	✔
Nessus Scan	✔

Advanced Settings

Working with Nessus Scan Output
Scanning Issues

🎧 Nessus Skills Assessment

OpenVAS

Getting Started with OpenVAS
OpenVAS Scan
Exporting The Results

🎧 OpenVAS Skills Assessment

Reporting

Reporting

My Workstation

O F F L I N E

▶ Start Instance

🔄 / 1 spawns left

